

TMM4128:

Machine Learning for Engineers

Andrei Lobov

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250313>

A2: Unsupervised learning on time series data

You are to implement a solution for anomalies detection in time series data. This can be used to monitor industrial equipment, detect anomalies and call human operators for checking the situation closer. Take a time series dataset made for any **operating machine / equipment**. Check if and which anomalies can be detected in the dataset.

Follow the ML project checklist. **Evaluate and report on each phase of the checklist. Compare performance** of different unsupervised learning techniques. Why do some perform better than others? How would you suggest to use the solution in online mode, i.e. getting real-time data from a machine?

Check for datasets here:

* <https://www.kaggle.com/>

Outline : Training Deep Neural Networks (code [samples](#))

- Definitions / Motivation
- Vanishing/Exploding Gradients Problems
- Reusing Pretrained Layers
- Faster Optimizers
- Avoiding Overfitting Through Regularization
- Summary and Practical Guidelines

—

Source: Aurelien Geron, “Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems”, 2019 ([Available online](#)) / **Chapter 11**

Definition / Motivation

What if a complex problem of detecting hundreds of types of object in high-resolution images? → Much deeper DNN (>> 10 layers with hundreds of neurons and 100K connections).

This gives several challenges: i) vanishing gradients problem (or exploding gradients problem) in DNN: lower layers very hard to train; ii) not enough training data for such a large network / too costly to label; iii) training may be extremely slow; iv) model with millions of parameters would severely risk overfitting the training set, especially if there are not enough training instances, or they are too noisy.

Vanishing/Exploding Gradients Problems

Vanishing/Exploding Gradients Problems

- Backpropagation algorithm goes from the output layer to the input layer, propagating the error gradient on the way.
- Once the algorithm has computed the gradient of the cost function with regards to each parameter in the network, it uses these gradients to update each parameter with a Gradient Descent step.
- Gradients often get smaller and smaller as the algorithm progresses down to the lower layers.
- Gradient Descent update leaves the lower layer connection weights virtually unchanged, and **training never converges to a good solution** (*vanishing gradients* problem.)
- The opposite can happen: the gradients can grow bigger and bigger, so many **layers get insanely large weight updates and the algorithm diverges** (exploding gradients problem).

Vanishing/Exploding Gradients Problems (1)

“Understanding the difficulty of training deep feedforward neural networks”

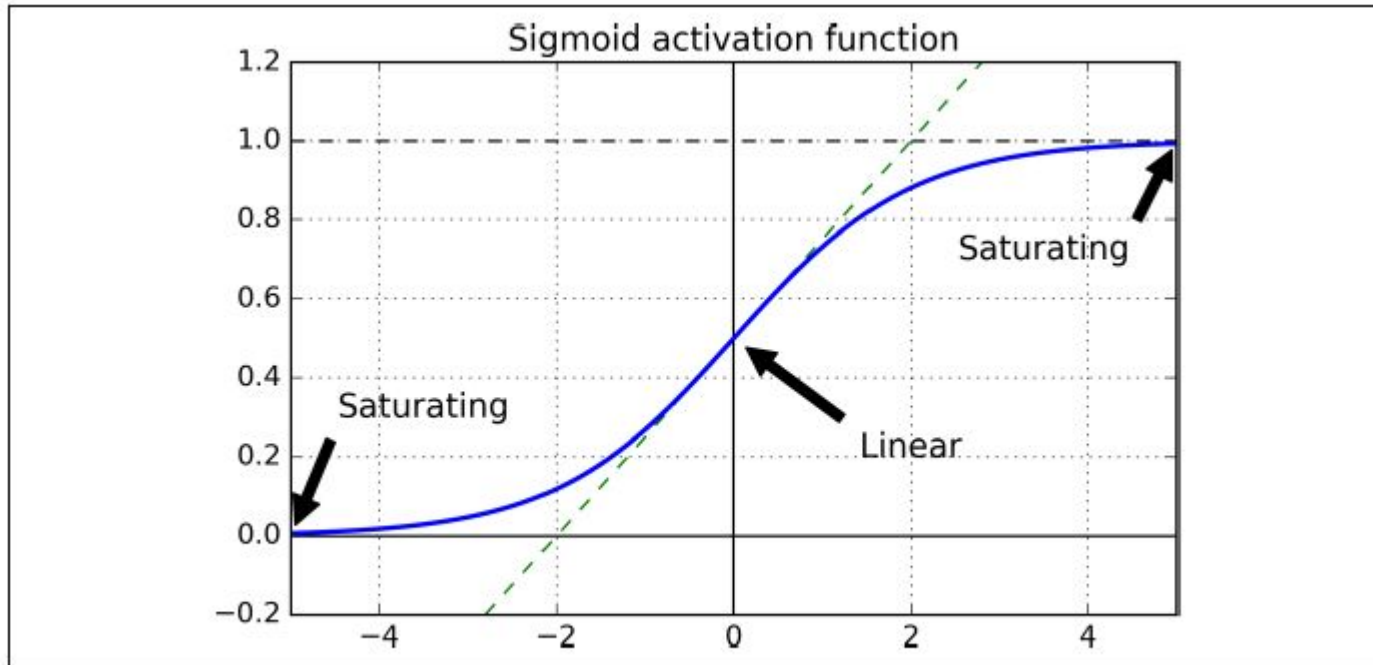


Figure 11-1. Logistic activation function saturation

Vanishing/Exploding Gradients Problems (2)

- Combination of the popular (2010) logistic sigmoid activation function and the weight initialization technique, namely random initialization using a normal distribution with a mean of 0 and a standard deviation of 1.
- It was showed that with this activation function and this initialization scheme, the **variance of the outputs of each layer is much greater than the variance of its inputs**.
- Going forward in the network, the variance keeps increasing after each layer until the activation function saturates at the top layers. This is made worse by the fact that the logistic function has a mean of 0.5, not 0 (the hyperbolic tangent function has a mean of 0 and behaves slightly better than the logistic function in deep networks).

Vanishing/Exploding Gradients Problems (3)

- Logistic activation function (see Figure 11-1): when inputs become large (negative or positive), the function saturates at 0 or 1, with a derivative extremely close to 0.
- Thus when backpropagation kicks in, it has virtually no gradient to propagate back through the network, and what little gradient exists keeps getting diluted as backpropagation progresses down through the top layers, so there is really nothing left for the lower layers.

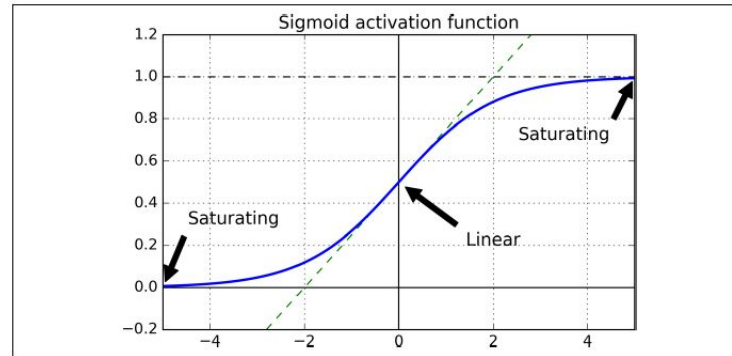


Figure 11-1. Logistic activation function saturation

Glorot and He Initialization (1)

- The signal to flow properly in both directions: in the forward direction when making predictions, and in the reverse direction when backpropagating gradients.
- Don't want the signal to die out, nor do we want it to explode and saturate.
- For the signal to flow properly, **the variance of the outputs of each layer to be equal to the variance of its inputs**, and we also need the **gradients to have equal variance before and after flowing through a layer** in the reverse direction.

Glorot and He Initialization (2)

- Not possible to guarantee both unless the layer has an equal number of inputs and neurons (these numbers are called the fan-in and fan-out of the layer).
- A good compromise to work very well in practice: the connection weights of each layer must be initialized randomly as follows:

Equation 11-1. Glorot initialization (when using the logistic activation function)

Normal distribution with mean 0 and variance $\sigma^2 = \frac{1}{fan_{avg}}$

Or a uniform distribution between $-r$ and $+r$, with $r = \sqrt{\frac{3}{fan_{avg}}}$

where $fan_{avg} = (fan_{in} + fan_{out})/2$.

Glorot and He Initialization (3)

Using Glorot initialization can speed up training considerably, and it is one of the tricks that led to the current success of Deep Learning.

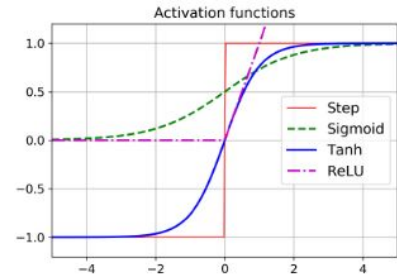
See [also](#).

Table 11-1. Initialization parameters for each type of activation function

Initialization	Activation functions	σ^2 (Normal)
Glorot	None, Tanh, Logistic, Softmax	$1 / fan_{avg}$
He	ReLU & variants	$2 / fan_{in}$
LeCun	SELU	$1 / fan_{in}$

By default, Keras uses Glorot initialization with a uniform distribution. Can be changed to He initialization by **kernel_initializer="he_uniform"** or **kernel_initializer="he_normal"**

Nonsaturating Activation Functions (1)



- Driven by Mother Nature: Sigmoid was believed to be the excellent choice as in biological neurons.
- However, other activation functions behave much better in deep neural networks.
 - the **ReLU** activation function: it does not saturate **for positive values** (and also it is quite fast to compute).
 - ReLU is not perfect - the *dying ReLUs*: during training, some neurons effectively die, meaning they stop outputting anything other than 0. In some cases, you may find that half of your network's neurons are dead, especially if you used a large learning rate.
 - A neuron dies when its weights get tweaked in such a way that the weighted sum of its inputs are negative for all instances in the training set. When this happens, it just keeps outputting 0s, and gradient descent does not affect it anymore since the gradient of the ReLU function is 0 when its input is negative.

Nonsaturating Activation Functions (2)

$$\text{LeakyReLU}_\alpha(z) = \max(\alpha z, z)$$

- **Leaky ReLU**: a variant of the ReLU function.
- α defines how much the function “leaks”: it is the slope of the function for $z < 0$, and is typically set to 0.01. This small slope ensures that leaky ReLUs never die.
- ReLU variants comparison: leaky variants always outperformed the strict ReLU activation function.
 - *Randomized leaky ReLU* (RRReLU): α is picked randomly in a given range during training, and it is fixed to an average value during testing.
 - *Parametric leaky ReLU* (PReLU), where α is authorized to be learned during training (instead of being a hyperparameter, it becomes a parameter that can be modified by backpropagation like any other parameter).

Nonsaturating Activation Functions (3)

$$\text{LeakyReLU}_{\alpha}(z) = \max(\alpha z, z)$$

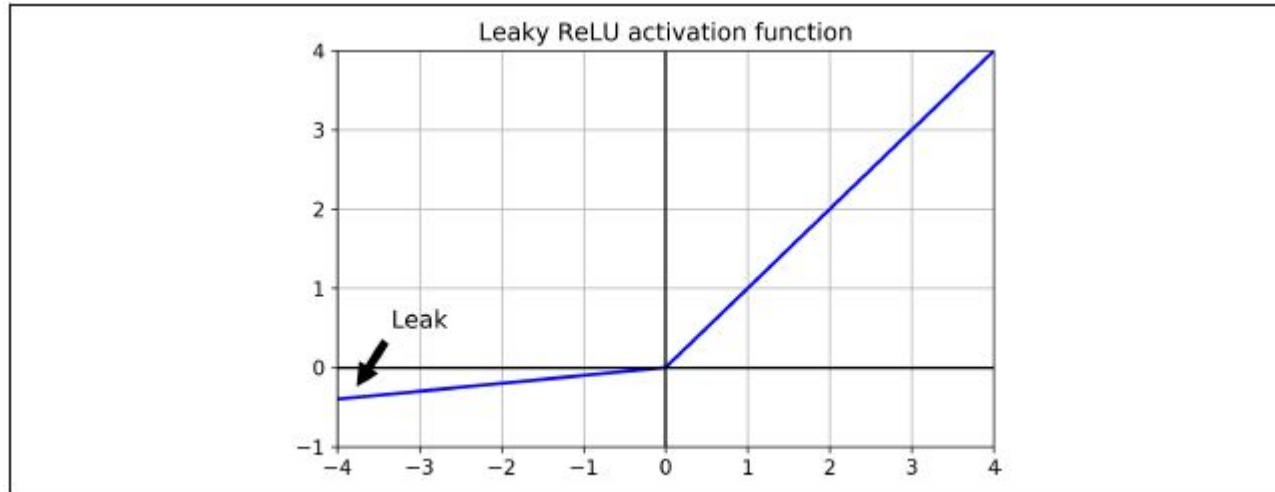


Figure 11-2. Leaky ReLU

Nonsaturating Activation Functions (4)

Exponential linear unit (ELU): can outperform the ReLU variants, training time can be reduced.

Equation 11-2. ELU activation function

$$\text{ELU}_{\alpha}(z) = \begin{cases} \alpha(\exp(z) - 1) & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$$

```
def elu(z, alpha=1):  
    return np.where(z < 0, alpha * (np.exp(z) - 1), z)
```

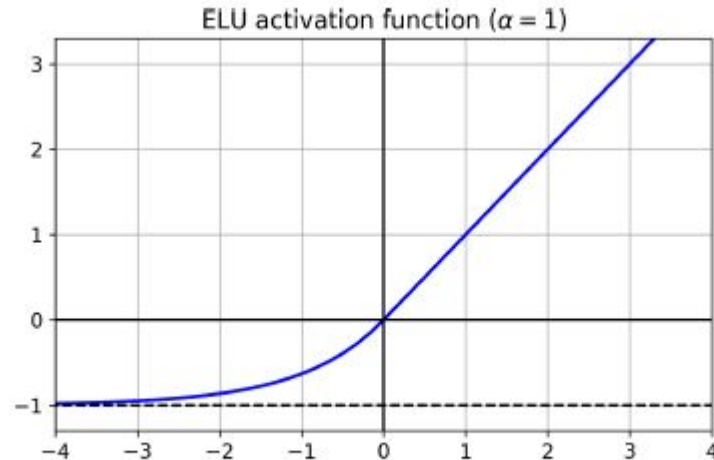


Figure 11-3. ELU activation function

Nonsaturating Activation Functions (5)

Exponential linear unit (ELU):

- Takes on negative values when $z < 0$, which allows the unit to have an average output closer to 0. So, helps alleviate the vanishing gradients problem. The hyperparameter α defines the value that the ELU function approaches when z is a large negative number: usually set to 1, but can be tweaked and tested with other values.
- Has a nonzero gradient for $z < 0$ to avoid the dead neurons problem.
- if α is equal to 1 then the function is smooth everywhere, including around $z = 0$, which helps speed up Gradient Descent, as it does not bounce as much left and right of $z = 0$.

Nonsaturating Activation Functions (6)

Exponential linear unit (ELU):

- It is slower to compute than the ReLU and its variants (due to the use of the exponential function), but **during training this is compensated by the faster convergence rate**. However, at test time an ELU network will be slower than a ReLU network.

SELU (scaled ELU) activation function: often outperforms other activation functions very significantly for such neural nets (especially deep ones).

```
def selu(z, scale=scale_0_1, alpha=alpha_0_1):  
    return scale * elu(z, alpha)
```

Nonsaturating Activation Functions (7)

SELU (scaled ELU) activation function: often outperforms other activation functions very significantly for such neural nets (especially deep ones).

- The input features must be standardized (mean 0 and standard deviation 1).
- Every hidden layer's weights must also be initialized using LeCun normal initialization.
`kernel_initializer="lecun_normal"`.
- The network's architecture must be sequential. Unfortunately, if you try to use SELU in non-sequential architectures, such as recurrent networks or networks with skip connections (i.e., connections that skip layers, such as in wide & deep nets), self-normalization will not be guaranteed, so SELU will not necessarily outperform other activation functions.
- Guaranteed self-normalization if all layers are dense. However, in practice the SELU activation function seems to work great with convolutional neural nets as well.

Which activation function to use for the hidden layers of DNN?

Generally: SELU > ELU > leaky ReLU (and its variants) > ReLU > tanh > logistic.

```
leaky_relu = keras.layers.LeakyReLU(alpha=0.2)
layer = keras.layers.Dense(10, activation=leaky_relu,
                             kernel_initializer="he_normal")
```

```
layer = keras.layers.Dense(10, activation="selu",
                             kernel_initializer="lecun_normal")
```

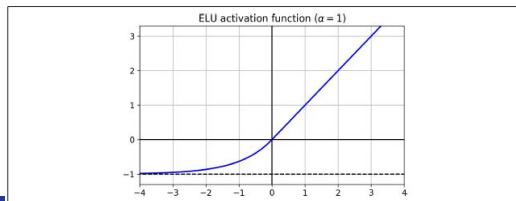


Figure 11-3. ELU activation function

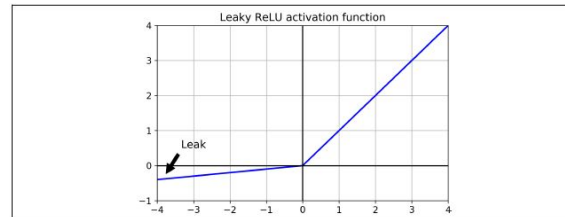
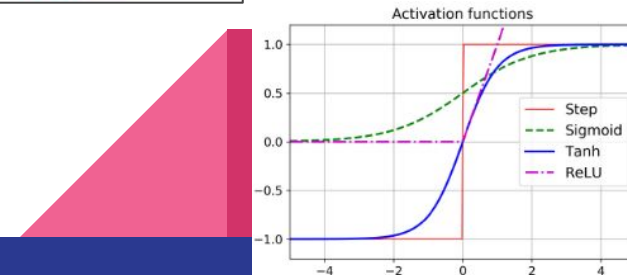


Figure 11-2. Leaky ReLU



Batch Normalization (1)

- ELU (or any variant of ReLU) can significantly reduce the vanishing/exploding gradients problems at the beginning of training.
- But no guarantee that these problems won't come back during training.
- So, Batch Normalization (BN) technique.
 - Two new parameter vectors per layer: one for **scaling**, the other for **shifting**.
 - So, this operation lets the model learn the optimal scale and mean of each of the layer's inputs.

Batch Normalization (2)

- μ_B is the vector of input means, evaluated over the whole mini-batch B (one mean per input).
- σ_B is the vector of input standard deviations, evaluated over the B (one standard deviation per input).
- m_B is the number of instances in the mini-batch.
- $\hat{\mathbf{x}}^{(i)}$ is the vector of zero-centered and normalized inputs for instance i .
- γ is the output scale parameter vector for the layer (one scale parameter per input).
- $\otimes$ represents element-wise multiplication (each input is multiplied by its corresponding output scale parameter).
- β is the output shift (offset) parameter vector for the layer (one offset parameter per input). Each input is offset by its corresponding shift parameter.
- ϵ is a tiny number to avoid division by zero (typically 10^{-5}). This is called a smoothing term.
- $\mathbf{z}^{(i)}$ is the output of the BN operation: it is a rescaled and shifted version of the inputs.

Equation 11-3. Batch Normalization algorithm

$$1. \quad \mu_B = \frac{1}{m_B} \sum_{i=1}^{m_B} \mathbf{x}^{(i)}$$

$$2. \quad \sigma_B^2 = \frac{1}{m_B} \sum_{i=1}^{m_B} (\mathbf{x}^{(i)} - \mu_B)^2$$

$$3. \quad \hat{\mathbf{x}}^{(i)} = \frac{\mathbf{x}^{(i)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$4. \quad \mathbf{z}^{(i)} = \gamma \otimes \hat{\mathbf{x}}^{(i)} + \beta$$

Batch Normalization (3)

BatchNormalization

```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(300, activation="elu", kernel_initializer="he_normal"),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(100, activation="elu", kernel_initializer="he_normal"),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(10, activation="softmax")
])
```

```
>>> model.summary()
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
flatten_3 (Flatten)	(None, 784)	0
batch_normalization_v2 (Batch Normalization)	(None, 784)	3136
dense_50 (Dense)	(None, 300)	235500
batch_normalization_v2_1 (Batch Normalization)	(None, 300)	1200
dense_51 (Dense)	(None, 100)	30100
batch_normalization_v2_2 (Batch Normalization)	(None, 100)	400
dense_52 (Dense)	(None, 10)	1010
Total params: 271,346		
Trainable params: 268,978		
Non-trainable params: 2,368		

```
>>> [(var.name, var.trainable) for var in model.layers[1].variables]
[('batch_normalization_v2/gamma:0', True),
 ('batch_normalization_v2/beta:0', True),
 ('batch_normalization_v2/moving_mean:0', False),
 ('batch_normalization_v2/moving_variance:0', False)]
```

Gradient Clipping

Another popular technique to lessen the exploding gradients problem is to clip the gradients during backpropagation so that they **never exceed some threshold**.

Most often used in recurrent neural networks (RNN).

```
optimizer = keras.optimizers.SGD(clipvalue=1.0)  
model.compile(loss="mse", optimizer=optimizer)
```


Reusing Pretrained Layers

Reusing Pretrained Layers (1)

- Not a good idea to train a very large DNN from scratch.
- Instead, try to find / use an existing neural network that accomplishes a similar task.
- Transfer learning: reuse the lower layers of that network.
- It speeds up training considerably, and requires much less training data.

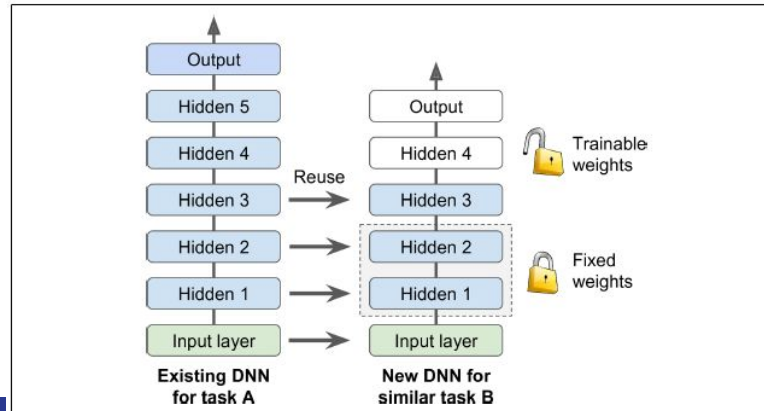


Figure 11-4. Reusing pretrained layers

Reusing Pretrained Layers (2)

- However, usually requires to add a preprocessing step to resize inputs to the size expected by the original model. (More generally, transfer learning will work best when the inputs have similar low-level features).
- The output layer of the original model should usually be replaced since it is most likely not useful at all for the new task, and it may not even have the right number of outputs for the new task.
- The upper hidden layers of the original model are less likely to be as useful as the lower layers, since the high-level features that are most useful for the new task may differ significantly from the ones that were most useful for the original task. So, **to find the right number of layers to reuse.**

Reusing Pretrained Layers (3)

- Try freezing all the reused layers first (i.e., make their weights non-trainable, so gradient descent won't modify them), then train the model and see how it performs.
- Then try unfreezing one or two of the top hidden layers to let backpropagation tweak them and see if performance improves.
- The more training data you have, the more layers you can unfreeze. It is also useful to reduce the learning rate when you unfreeze reused layers: this will avoid wrecking their fine-tuned weights.

Reusing Pretrained Layers (4)

- If still a bad performance, and there is little training data, try dropping the top hidden layer(s) and freeze all remaining hidden layers again.
- Iterate until you find the right number of layers to reuse.
- If you have plenty of training data, you may try replacing the top hidden layers instead of dropping them, and even add more hidden layers.

Transfer Learning With Keras (1)

- Fashion MNIST dataset for 8 classes.
 - Someone built and trained a Keras model on that set and got reasonably good performance (>90% accuracy) : **Model A**.
 - A new task: you have images of **sandals** and **shirts**, and you want to train a **binary classifier** (positive=shirts, negative=sandals). The dataset is quite small, you only have **200 labeled images**. When you train a new model for this task (**model B**), with the same architecture as model A, it performs reasonably well (97.2% accuracy), but since it's a much easier task (there are just 2 classes), you were hoping for more. → Transfer Learning?

```
model_A = keras.models.load_model("my_model_A.h5")  
model_B_on_A = keras.models.Sequential(model_A.layers[:-1])  
model_B_on_A.add(keras.layers.Dense(1, activation="sigmoid"))
```

reusing layers (except output)

vs.

```
model_A_clone = keras.models.clone_model(model_A)  
model_A_clone.set_weights(model_A.get_weights())
```

cloning

Transfer Learning With Keras (2)

Train **model_B_on_A** for task B, but since the new output layer was initialized randomly, it will make large errors, at least during the first few epochs, so there will be large error gradients that may wreck the reused weights. To avoid this, one approach is to freeze the reused layers during the first few epochs, giving the new layer some time to learn reasonable weights. To do this, simply set every layer's trainable attribute to False and compile the model:

```
for layer in model_B_on_A.layers[:-1]:  
    layer.trainable = False  
model_B_on_A.compile(loss="binary_crossentropy", optimizer="sgd",  
                     metrics=["accuracy"])
```

Transfer Learning With Keras (3)

Next, train the model for a few epochs, then unfreeze the reused layers (which requires compiling the model again) and continue training to fine-tune the reused layers for task B. After unfreezing the reused layers, it is usually a good idea to reduce the learning rate, once again to avoid damaging the reused weights:

```
history = model_B_on_A.fit(X_train_B, y_train_B, epochs=4,
                           validation_data=(X_valid_B, y_valid_B))

for layer in model_B_on_A.layers[:-1]:
    layer.trainable = True

optimizer = keras.optimizers.SGD(lr=1e-4) # the default lr is 1e-3
model_B_on_A.compile(loss="binary_crossentropy", optimizer=optimizer,
                     metrics=["accuracy"])
history = model_B_on_A.fit(X_train_B, y_train_B, epochs=16,
                           validation_data=(X_valid_B, y_valid_B))
```


Transfer Learning With Keras (4)

This model's test accuracy is 99.25%, which means that transfer learning reduced the error rate from 2.8% down to almost 0.7%. (by a factor of 4).

```
>>> model_B_on_A.evaluate(X_test_B, y_test_B)  
[0.06887910133600235, 0.9925]
```

Well, that was a 'cheat'. Transfer learning does not work very well with small dense networks: **it works best with deep convolutional neural networks.**

Unsupervised Pretraining (1)

- A complex task with not much labeled training data, no previously trained model on a similar task. Expensive labeling? → unsupervised pretraining.
- Cheaper to gather unlabeled training examples, but quite expensive to label them.
- Train the layers one by one, starting with the lowest layer and then going up, using an unsupervised feature detector algorithm such as Restricted Boltzmann Machines (RBMs) or autoencoders.
- Each layer is trained on the output of the previously trained layers (all layers except the one being trained are frozen).
- Once all layers have been trained this way, add the output layer for the task, and fine-tune the final network using supervised learning (i.e., with the labeled training examples). Unfreeze all the pretrained layers, or just some of the upper ones.

Unsupervised Pretraining (2)

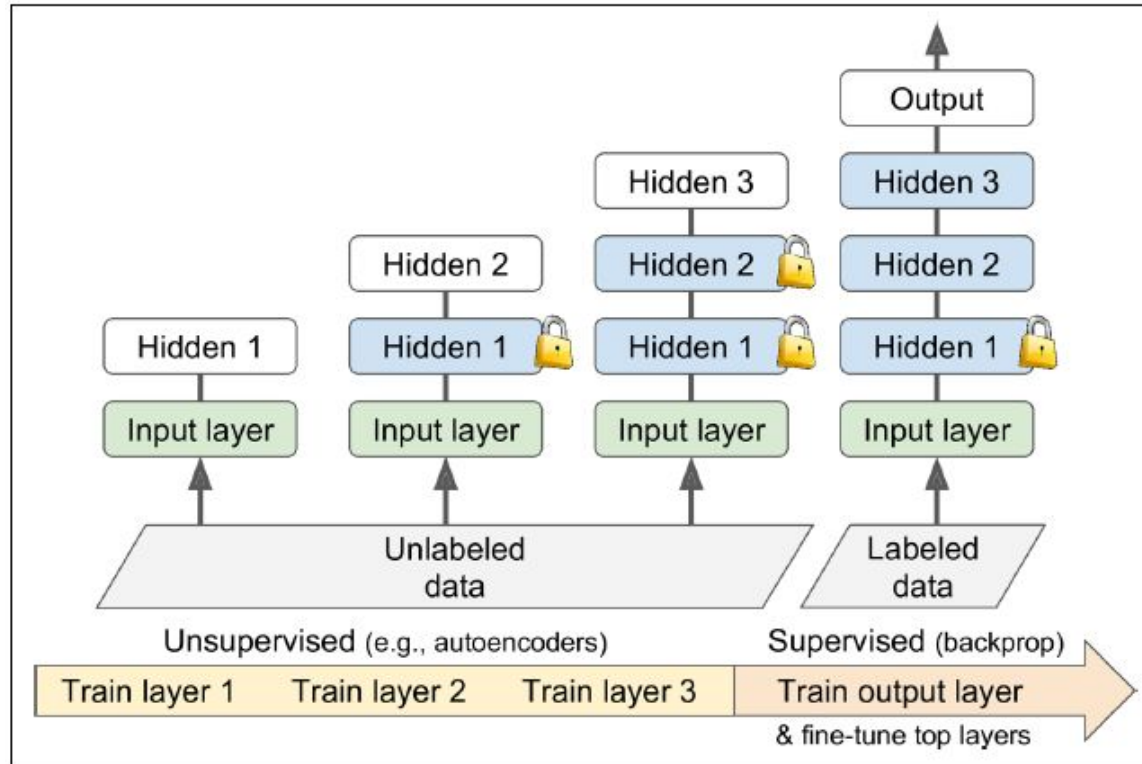


Figure 11-5. Unsupervised pretraining

Pretraining on an Auxiliary Task (1)

- No much labeled training data: one last option is to train a first neural network on an auxiliary task for which you can easily obtain or generate labeled training data.
- Then reuse the lower layers of that network for your actual task. The first neural network's lower layers will learn feature detectors that will likely be reusable by the second neural network.
- Example: a system to recognize faces with a few pictures of each individual - so, not enough to train a good classifier. Gathering hundreds of pictures of each person is not practical. But, you could gather a lot of **pictures of random people** on the web and train a first neural network **to detect whether or not two different pictures feature the same person**. So, learning good feature detectors for faces.

Pretraining on an Auxiliary Task (2)

- For natural language processing (NLP), millions of text documents available for download to automatically generate labeled data.
- Example: randomly mask out some words and train a model to predict what the missing words are (e.g., it should predict that the missing word in the sentence “What ___ you saying?” is probably “are” or “were”). If you can train a model to reach good performance on this task, then it will already know quite a lot about language, and you can certainly reuse it for your actual task, and fine-tune it on your labeled data.

Takeaways

- Alternative activation functions to help addressing Vanishing/Exploding Gradients Problems.
- Organisation of transfer learning to reuse pretrained models (some selected bits of those).